GST Compliance & Smart Billing Portal

GitHub Repository Walkthrough

Repository: https://github.com/chetank23/GST-INVOICE-AI.git

Document Date: April 2026

1. Project Overview

AIAD (AI-Assisted Document) is a production-ready Frappe v15 / ERPNext application built specifically for Indian GST compliance automation. The project integrates AI-powered billing, voice command input, OCR scanning, and a multi-tenant portal suite into a single Dockerised stack.

The application is hosted on GitHub and can be cloned and run entirely using Docker Desktop on any Windows or Linux machine. The GitHub repository URL is:

```
https://github.com/Fearoff5214/AIAD.git
```

1.1 Core Features

- AI-powered GST Copilot using local Ollama (DeepSeek-R1:7b) — answers GST queries in plain English
- Voice Billing — speak items in Hindi, English, or Hinglish to generate GST invoices
- OCR Bill Scanning — upload a photo of a physical bill and extract line items automatically
- Multi-tenant portal with self-service registration for manufacturers and wholesalers
- GST Compliance Demo — live validator comparing correct vs incorrect invoice formats
- Full ERPNext Desk available via /app for advanced ERP usage

1.2 Technology Stack

Layer	Technology	Purpose
Backend Framework	Frappe v15 / ERPNext	Core ERP and Python API layer
Database	MariaDB 10.6	Persistent data storage
Cache & Queues	Redis 6.2	Job queues and caching
AI / LLM	Ollama + DeepSeek-R1:7b	Local language model for GST copilot
OCR Engine	Tesseract OCR + pytesseract	Bill image text extraction
Frontend	NGINX + Vue.js components	Reverse proxy and UI components
Containerisation	Docker + Docker Compose	Full-stack orchestration
Custom App	india_compliance/ (Python)	All GST and billing logic

2. Cloning the Repository

Follow the steps below to get a local copy of the AIAD repository onto your machine.

2.1 Prerequisites

- Git installed (git --version to verify)
- Docker Desktop installed and running
- Ollama installed on the host machine
- Python 3.x available (for the startup script)

2.2 Clone Command

Open a terminal (PowerShell on Windows or bash on Linux/Mac) and run:

```
git clone https://github.com/Fearoff5214/AIAD.git
```

This creates an AIAD/ folder in your current directory containing the full project.

2.3 Pull the AI Model

Before starting the application, pull the DeepSeek model that powers the GST Copilot:

```
ollama pull deepseek-r1:7b
```

This is a one-time step. The model runs entirely on your local machine — no cloud API keys are needed.

3. Repository Structure

The repository root contains configuration files and the main custom Frappe application directory. Below is a map of the key files and folders:

3.1 Root-Level Files

File / Folder	Purpose
Dockerfile	Builds the custom Docker image (adds Tesseract OCR on top of ERPNext v15)
docker-compose.yml	Defines and orchestrates all 9 Docker services
run_fearoff_aiad.bat	One-click Windows batch script to start the entire stack
start_frappe_server.py	Python orchestrator with startup flags for finer control
bootstrap_primary_admin.py	Script to seed the primary admin account on first run
india_compliance/	The entire custom Frappe application (Python + JS + JSON)
README.md	Quick-start guide and feature overview
SETUP_AND_STARTUP.md	Detailed setup, flags, and deployment notes
STARTUP_TROUBLESHOOTING.md	Common errors and their fixes
pyproject.toml / setup.py	Python package configuration for india_compliance app
package.json	Node.js dependencies for frontend assets

3.2 India_compliance/ Application Layout

The india_compliance/ directory is the heart of the project. It is a standard Frappe app with the following sub-modules:

Sub-module	Key Contents	Description
gst_india/api/	voice_billing.py, ocr_billing.py, gst_copilot.py, portal_management.py	All API endpoints for billing, OCR, and AI
gst_india/api_classes/	e_invoice.py, e_waybill.py, returns.py	Classes for GST API integrations (e-Invoice, e-Waybill)
gst_india/doctype/	e_invoice_log, e_waybill_log, gst_settings, gstr_3b_report	Frappe DocTypes for GST data
gst_india/overrides/	sales_invoice.py, purchase_invoice.py, party.py, etc.	Frappe document hooks to add GST logic
gst_india/report/	GSTR-1, GST Sales Register, HSN Summary, etc.	Built-in GST reports
gst_india/utils/	e_invoice.py, e_waybill.py, transaction_data.py	Utility functions for GST calculations
gst_india/data/	gst_compliance_knowledge.md, hsn_codes.json	Knowledge base and HSN master data
cleartax_integration/	API/, doctype/	ClearTax third-party integration layer
income_tax_india/	constants/, overrides/	Income tax and TDS-related customisations
www/	user-portal.html, smart-billing.html, gst-copilot.html, etc.	Web-accessible portal HTML pages
public/js/	Vue.js components, bundle files	Frontend JavaScript and Vue SPA components
patches/post_install/	migration scripts	Database migration patches

4. Docker Architecture

The entire application stack is defined in docker-compose.yml and runs as 9 Docker services on an internal bridge network called frappe_network.

4.1 Services Overview

Service	Image / Build	Port	Role
backend	Custom (Dockerfile)	8000 (internal)	Gunicorn WSGI server running Frappe/ERPNext
frontend	Custom (Dockerfile)	8085 → 8080	NGINX reverse proxy — the only externally exposed port
db	mariadb:10.6	3306 (internal)	MySQL-compatible database
redis-cache	redis:6.2-alpine	6379 (internal)	In-memory cache layer
redis-queue	redis:6.2-alpine	6379 (internal)	Job queue for background tasks
scheduler	Custom (Dockerfile)	—	Runs Frappe's scheduled tasks (bench schedule)
queue-long	Custom (Dockerfile)	—	Background worker for long-running jobs
queue-short	Custom (Dockerfile)	—	Background worker for short/quick jobs
websocket	Custom (Dockerfile)	9000 (internal)	Socket.IO server for real-time updates

4.2 Custom Dockerfile

The custom Docker image is built on top of frappe/erpnext:v15.0.0 and adds:

- Tesseract OCR and Tesseract English language pack
- NGINX patch to accept localhost as a valid server name
- Python dependencies: pytesseract, Pillow, numpy
- The full india_compliance app copied into apps/

4.3 Volumes

- sites — Frappe site files (databases connections, installed apps list)
- logs — Application logs
- db-data — MariaDB persistent data
- redis-queue-data — Persistent queue data

4.4 Environment Variables

Key environment variables injected into backend and related services:

Variable	Value	Purpose
DB_HOST	db	Hostname of MariaDB container
MYSQL_ROOT_PASSWORD	admin	Database root password
OLLAMA_BASE_URL	http://host.docker.internal:11434	Connects backend to host Ollama
OLLAMA_MODEL	deepseek-r1:7b	Model name for GST Copilot
OLLAMA_TIMEOUT_SECONDS	300	Max wait time for AI responses
FRAPPE_SITE_NAME_HEADER	frontend	Routes HTTP requests to the correct Frappe site

5. Starting the Application

5.1 One-Click Startup (Windows)

Double-click run_fearoff_aiad.bat or run it from PowerShell:

```
run_fearoff_aiad.bat
```

5.2 Manual Startup

Navigate to the project folder and bring up all Docker services:

```
cd AIAD
docker compose up -d
```

Wait approximately 60 seconds for Frappe to initialise, then open your browser at:

```
http://localhost:8085
```

5.3 Python Startup Script (Advanced)

For finer-grained control, use the Python orchestrator:

```
python start_frappe_server.py
```

Available flags:

- --no-build — Skip rebuilding the Docker image
- --skip-migrate — Skip Frappe site migration (faster restart)
- --skip-ollama-pull — Do not pull the Ollama model
- --skip-ollama-service — Do not auto-start Ollama

--follow-logs — Stream Docker container logs after startup

6. Portal Pages & Navigation

All portal pages share a unified dark-teal navigation bar and are accessible at localhost:8085 once the stack is running.

URL Path	Page Name	Description
/user-portal	User Dashboard	Sector-tailored dashboard with GST feature tiles
/smart-billing	Smart Billing	Voice + OCR billing with Hinglish support
/gst-copilot	GST Copilot	AI chat assistant powered by Ollama DeepSeek
/gst-demo	GST Demo	Interactive correct vs incorrect invoice comparison
/user-register	Register	Self-service business account registration
/admin-user-management	Admin Panel	Tenant management, user admin, usage analytics
/app	ERPNext Desk	Full Frappe/ERPNext ERP interface

7. Key API Endpoints

7.1 Voice Billing

POST — Parse spoken/typed item descriptions into structured invoice data:

```
POST /api/method/india_compliance.gst_india.api.voice_billing.process_voice
Body: { "text": "2 kg rice 100 and 1 oil 150" }
```

POST — Confirm and save a voice-generated invoice:

```
POST /api/method/india_compliance.gst_india.api.voice_billing.confirm_voice_invoice
```

GET — Auto-suggest item names and HSN codes:

```
GET
/api/method/india_compliance.gst_india.api.voice_billing.get_item_suggestions?query=ri
ce
```

7.2 OCR Billing

POST — Process a base64-encoded bill image and extract line items:

```
POST /api/method/india_compliance.gst_india.api.ocr_billing.process_ocr
Body: { "image_base64": "...", "filename": "bill.jpg" }
```

POST — Confirm and save an OCR-extracted invoice:

```
POST /api/method/india_compliance.gst_india.api.ocr_billing.confirm_ocr_invoice
```

7.3 GST Copilot

POST — Ask a GST compliance question:

```
POST /api/method/india_compliance.gst_india.api.gst_copilot.gst_copilot_chat
Body: { "question": "What is the GST rate for rice?" }
```

GET — Fetch suggested questions for the copilot UI:

```
GET
/api/method/india_compliance.gst_india.api.gst_copilot.gst_copilot_suggested_questions
```

8. Demo Accounts

Three demo accounts are seeded during initial setup to allow immediate access to the portal without registration:

Email	Password	Role
parameshlogaa@gmail.com	PE42026	Primary Admin — full access
test_manufacturer@example.com	Test@12345	Manufacturing sector user
test_wholesaler@example.com	Test@12345	Wholesale sector user

Important: Change the primary admin password immediately in any production or shared deployment.

9. Troubleshooting

9.1 502 Bad Gateway

NGINX is not recognising localhost as a valid server name. Patch it manually:

```
docker exec -u root fearoff_aiad-frontend-1 sed -i 's/server_name
frontend;/server_name frontend localhost;/' /etc/nginx/conf.d/frappe.conf
docker exec fearoff_aiad-frontend-1 nginx -s reload
```

9.2 OCR Not Working

Verify Tesseract is installed inside the backend container:

```
docker exec fearoff_aiad-backend-1 tesseract --version
docker exec fearoff_aiad-backend-1 pip show pytesseract
```

9.3 GST Copilot Not Responding

Ensure Ollama is running on the host and the model is loaded:

```
ollama list
ollama run deepseek-r1:7b "hello"
```

9.4 Full Reset

If the application state has drifted or the site is corrupted:

```
docker compose down
docker volume rm fearoff_aiad_sites fearoff_aiad_db-data
docker compose up -d
```

This removes all persistent data and recreates the site from scratch.

9.5 404 on localhost:8085

The Frappe site alias mapping was not created. Re-run the startup script (start_frappe_server.py) to recreate the host routing.

10. GST & Compliance Features

10.1 Voice Billing Detail

The voice billing engine (voice_billing.py) accepts free-form text in English, Hindi, or Hinglish and parses it into structured invoice lines. The supported commands are:

Voice Command	Action
"2 kg rice 100 and 1 oil 150"	Adds two line items with quantity, unit, and price
"Remove last item" / "hatao"	Removes the last line item
"Clear all" / "saaf"	Clears all items from the current invoice
"Generate bill" / "bana"	Triggers invoice confirmation flow
"Show total" / "kitna"	Refreshes and displays the running total

The engine also auto-suggests HSN codes and GST rates for more than 60 commonly traded items.

10.2 OCR Pipeline

The OCR pipeline (ocr_billing.py) follows these stages:

- Image uploaded as base64 from the browser
- Preprocessing — contrast enhancement and noise reduction using Pillow and numpy
- Tesseract OCR runs and extracts raw text
- AI parsing stage cleans and structures the extracted text into invoice fields (GSTIN, date, line items, tax amounts)
- Validation checks GST rates and GSTIN format
- User reviews the result in an editable table before confirming

OCR output is never auto-saved — manual confirmation is always required.

10.3 GST Reports Available

- GSTR-1 — Outward supplies summary
- GSTR-3B Report — Monthly return data
- GST Sales Register and Purchase Register
- GST Itemised Sales and Purchase Registers
- HSN-wise Summary of Outward Supplies
- E-Invoice Summary